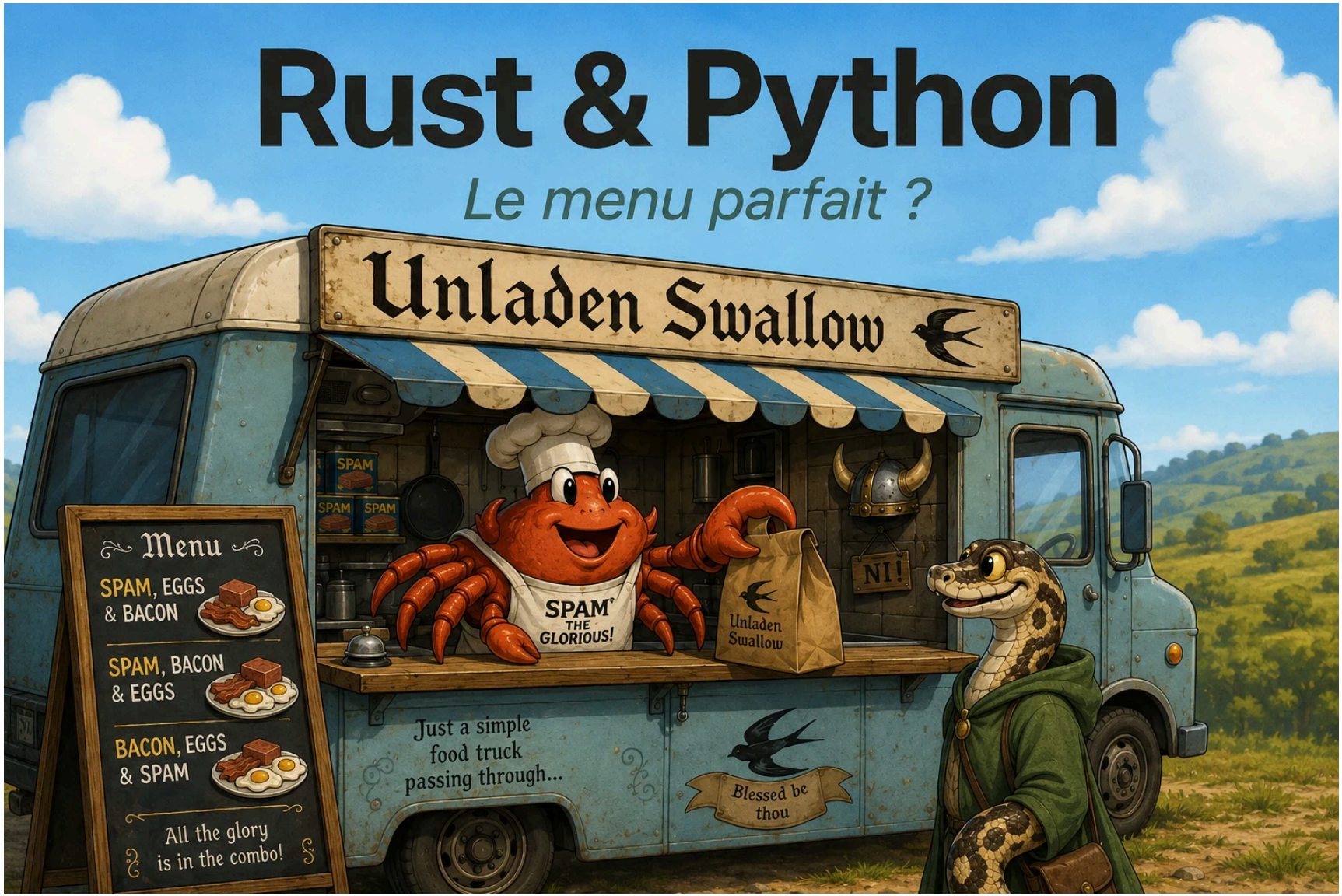

Rust & Python
Le menu parfait ?
Unladen Swallow
SPAM
SPAM
SPAM
Menu
SPAM, EGGS & BACON
SPAM, BACON & EGGS
BACON, EGGS & SPAM
All the glory is in the combo!
SPAM® THE GLORIOUS!
Unladen Swallow
NI!
Just a simple food truck passing through...
Blessed be thou

Mise en bouche

Pourquoi appeler du Rust depuis Python ?

$ whoami

$ whoami

Igor Laborie *Founding backend engineer*

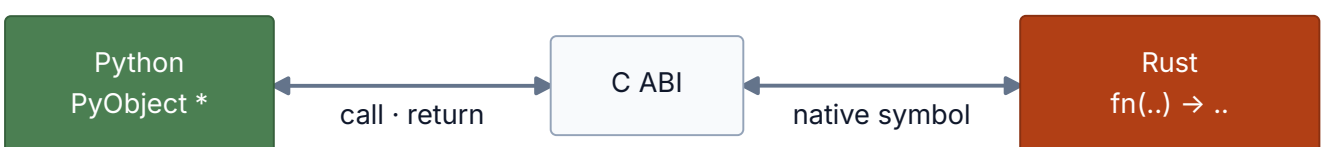
😍 Rust 🤔 Python

Le constat

- 🐍 Python peut appeler du **C** avec `ctypes`, `cffi`, C-API
- 🦀 Rust peut produire un binaire compatible avec l'ABI **C**

```rust
#[repr(C)]                    // Représentation mémoire C
pub struct Point { x: f64, y: f64 }

#[unsafe(no_mangle)]          // Le nom est préservé `norm`, sinon `_ZN4norm…`
pub extern "C" fn norm(p: *const Point) -> f64 { /* … */ }
```



Techniquement, ça marche C'est quoi le problème ?

Interopérabilité

Approche	Pour	Contre
`ctypes`	pas de compilation	lent, aucune sécurité
`cffi`	mode ABI ou API	une dépendance de plus
C-API CPython	perf. maximale	à faire manuellement
PyO3	Rust + perf C-API	compiler du Rust

PyO3: conversions de types, exceptions, docstrings, signature Maturin: build & wheels

Bidirectionel : Python → Rust, et Rust → Python

Quel intérêt ?

> Rust offers **power and precision** to go beyond Python's limits.
>
> – David Hewitt, mainteneur de PyO3

Fiabilité, performance , sûreté mémoire, concurrence

Polars, DataFusion, pydantic-core, tokenizers, orjson, ..

La stack Python qui va vite est déjà écrite en **Rust**

Ce que PyO3 doit réconcilier

	Python	Rust
Typage	dynamique, à l'exécution	statique, à la compilation
Objets	héritage, monkey-patching	structs + traits
Collections	`dict` hétérogène	`HashMap<K, V>` K et V fixer à la compilation
Erreurs	exceptions	`Result<T, E>`
Mémoire	refcount + GC	Ownership, pas de GC
Threads	GIL	protection enver les data race avec `Send` et `Sync`

Deux modèles très différents PyO3 fait la traduction

La mise en place

PyO3 + Maturin, et on écrit du code

Ce qu'on va servir

```Python
>>> import md
>>> md.to_html("**Bonjour**")
'<p><strong>Bonjour</strong></p>\n'
```

Une fonction **Python**, un moteur **Rust**.

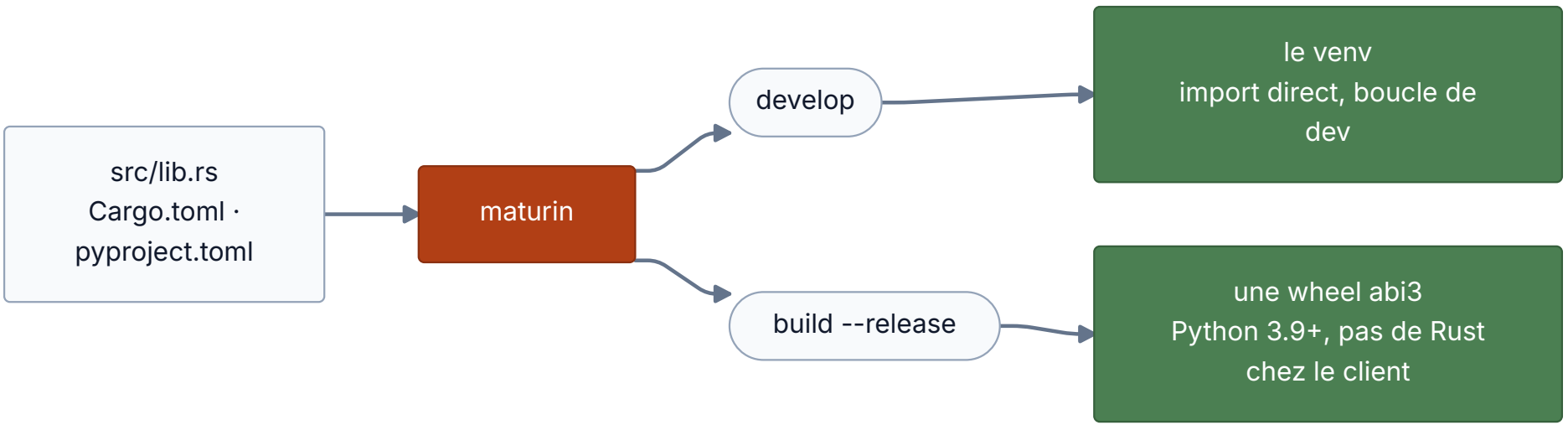
Deux fichiers, et c'est tout

```toml
# Cargo.toml
[lib]
crate-type = ["cdylib"] # C dynamic library (.so, .dll)

[dependencies]
pyo3 = "0.30"
```

```toml
# pyproject.toml
[build-system]
requires = ["maturin≥1.9,<2.0"]
build-backend = "maturin"
```

De la source à la wheel




```Shell
$ maturin develop          # compile + installe dans le venv
$ maturin build --release  # produit une wheel
```

Le résultat

Rust

```rust
/// Markdown module
#[pyo3::pymodule]
mod md {
    use pyo3::prelude::*;
    // ...

    /// Render markdown content into HTML
    #[pyfunction(signature = (md:"str", gfm = false))]
    pub fn to_html(md: &str, gfm: bool) -> PyResult<String> {
        let mut options = Options::default();
        let mut plugins = Plugins::default();
        // ...
        let result = comrak::markdown_to_html_with_plugins(md, &options, &plugins);
        Ok(result)
    }
}
```

RTFM: pyo3.rs

Dans l'autre sens

Un binaire **Rust** qui embarque un interpréteur **Python**.

```rust
fn main() -> PyResult<()> {
    Python::attach(|py| {
        let math = py.import("math")?;
        let pi: f64 = math.getattr("pi")?.extract()?;
        println!("π = {pi}");

        py.run(c"print('Bonjour depuis Python')", None, None)
    })
}
```

```toml
pyo3 = { version = "0.30", features = ["auto-initialize"] }
```

Et si on réécrivait Python ?

Monty : un interpréteur Python écrit en Rust, pour exécuter du code non fiable

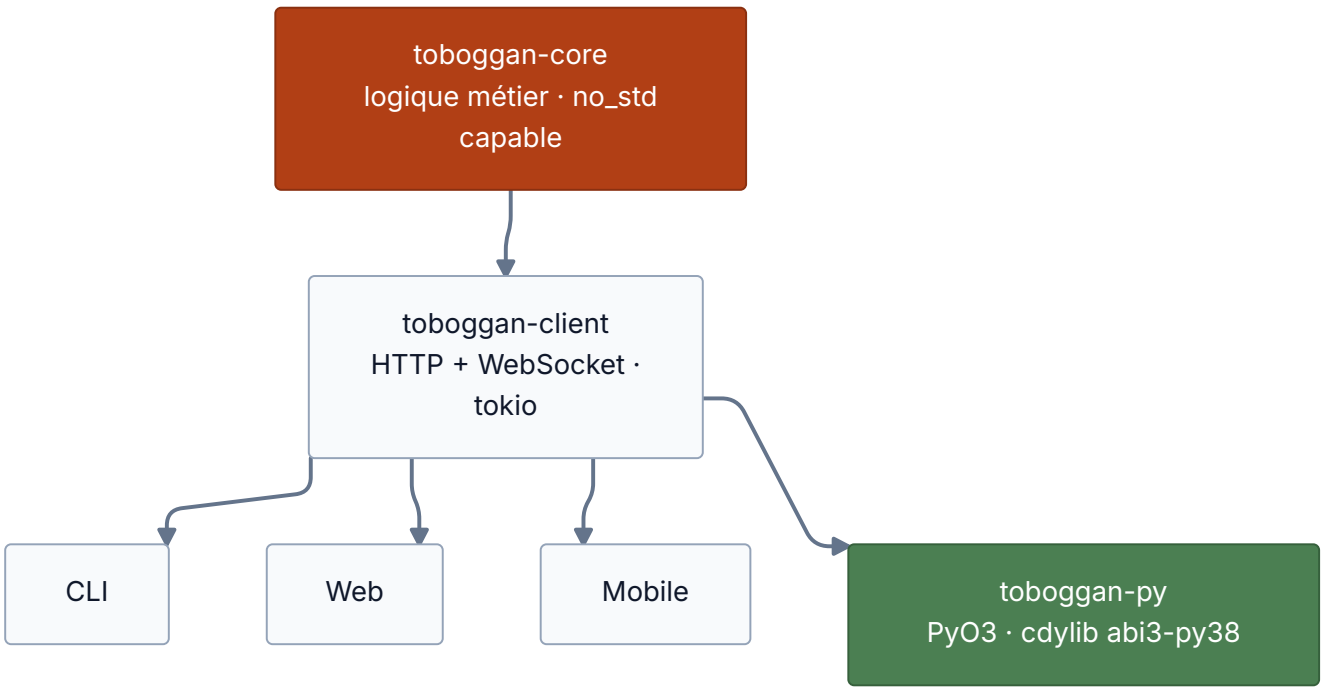
- **PyO3** embarque CPython : tout Python, tout l'écosystème
- **Monty** le remplace : un sous-ensemble de Python, isolé de l'hôte

En Python avec `pydantic-monty`… écrit avec PyO3

Le coup de feu

Deux projets, en production

Un cas réel : toboggan-py



Un même client, exposé en CLI, Web, Mobile et Python

> **Important**
>
> L'API Rust est *async* L'API Python est *sync* (par choix)

Rust async, Python sync

Rust

```rust
#[pyclass]
pub struct Toboggan {
    rt: Runtime,                // le runtime tokio
    api: TobogganApi,
    state: Arc<RwLock<TState>>,  // l'état
}
```

> ⚠️ **Warning**
>
> En appelant `block_on` dans chaque `#[pymethods]`, pas besoin de `await` en Python

Appel terminé, état à jour ?

```python
tbg.next()
tbg.state   # encore la slide précédente : 24 lectures sur 24
```

Avant : commande envoyée par WebSocket → retour immédiat → état reçu plus tard.

Après : `POST /api/command` → état appliqué → cache mis à jour → retour Python.

→ Une méthode synchrone doit tenir sa **promesse de cohérence**.

Le piège du GIL

```rust
// `block_on` verrouille le GIL pendant les I/O
let notif = self.rt.block_on(self.api.command(cmd))?;
```

```rust
// `detach` relache le lock pendant les I/O
let notif = py.detach(|| self.rt.block_on(self.api.command(cmd)))?;
```

```rust
// L'autre choix avec pyo3-async-runtimes
// En Python: await fetch(url)
#[pyfunction]
fn fetch(py: Python<'_>, url: String) -> PyResult<Bound<'_, PyAny>> {
    pyo3_async_runtimes::tokio::future_into_py(py, async move {
        let rsp = reqwest::get(&url).await.map_err(to_py)?;
        rsp.text().await.map_err(to_py)
    })
}
```

pyo3.rs/v0.30.0/async-await

Et l'event loop Python ?

`detach` libère les autres threads ; `await` **laisse avancer la boucle asyncio**.

Rust

```rust
#[pyfunction]
fn fetch(py: Python<'_>, url: String) -> PyResult<Bound<'_, PyAny>> {
    pyo3_async_runtimes::tokio::future_into_py(py, async move {
        let response = reqwest::get(&url).await.map_err(to_py)?;
        response.text().await.map_err(to_py)
    })
}
```

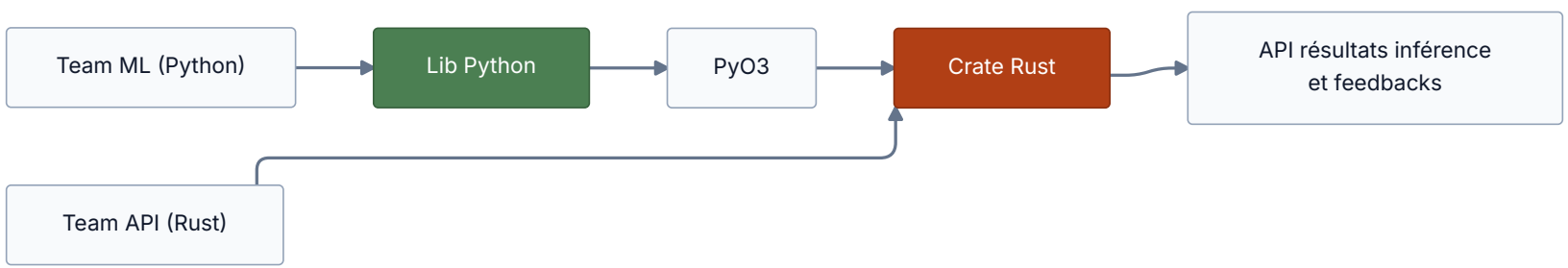
Python

```python
body = await fetch(url)
```

→ Choisir le contrat Python : **appel synchrone** ou **awaitable**.

Chez Wefox AI: une API client, deux équipes

Chez Wefox AI: micro service pour stocker les résultats d'inférence et leurs feedback



Implémentation partagée, une API adaptée à chaque équipe, on masque le détail d'implémentation

Télémétrie: comment propager la trace?



Ce qui ne traverse pas	Sens	Le pont
Les logs `tracing`	Rust → Python	`pyo3-log`
Le contexte de trace	Python → Rust	W3C TraceContext

Une requête, une trace



Pont	Sens	Mécanisme
Logs	Rust → Python	`pyo3-log`
Contexte de trace	Python → Rust	W3C TraceContext

Les logs passent

```rust
// tracing = { version = "0.1", features = ["log-always"] }
#[pymodule]
mod rusty_otel {
    #[pymodule_export]
    use super::render;

    #[pymodule_init]
    fn init(m: &Bound<'_, PyModule>) -> PyResult<()> {
        pyo3_log::init();    // log → logging Python
        telemetry::init();   // subscriber tracing + exporter OTel
        Ok(())
    }
}
```

Rust

```python
import rusty_otel   # appel de init
```

Python

```
INFO app         [trace=6a79…5545] appel du moteur Rust
INFO rusty_otel  [trace=6a79…5545] rendu de 53 octets de markdown
```

Text

Le contexte aussi

```rust
//  Récupère le context OTel de Python, et le passe en Rust
fn python_context(py: Python<'_>) -> PyResult<Context> {
    let carrier = PyDict::new(py);
    py.import("opentelemetry.propagate")?
        .call_method1("inject", (&carrier,))?;  // {"traceparent": "00-6a79…-01"}

    let carrier = carrier.extract::<HashMap<String, String>>()?;
    let context = global::get_text_map_propagator(|prop| prop.extract(&carrier));
    Ok(context)
}
```

```rust
let span = tracing::info_span!("render", bytes = input.len());
let parent = python_context(py).unwrap_or_default();
if let Err(err) = span.set_parent(parent) {
    tracing::warn!("contexte Python non rattaché : {err}");
}
// Tip: utiliser des macro_rules pour simplifier ce code
```

pyo3.rs/v0.30.0/ecosystem/tracing

L’addition

Ce que ça coûte, ce que ça rapporte

Ce qui coince

- Pas de génériques dans les classes: `#[pyclass] struct Container<T>`
- Pas de lifetime dans les classes: `#[pyclass] struct Container<'a>`
- Pas d'interop traits vers protocoles, par ex. `Iterator` ne devient pas itérable sans écrire manuellement les `__iter__` & `__next__`
- La génération des `.pyi` n'est pas automatique, Python typing hints, #5137
- Ne pas bloquer le GIL

Tips

- Réduire la surface d’interop
- Batcher, un appel pour 1 M d’éléments, pas 1 M d’appels
- `tool.uv.cache-keys` sur `src/**/*.rs` : `uv run` recompile tout seul
- `py.detach(...)` sur les sections CPU-bound et les I/O bloquantes si on `block_on`
- Tester des deux côtés avec `pytest` et `cargo test`
- `py-spy`: le flamegraph peut voir les frames Rust
- Faire attention aux mesures de performance: `--release`

Quand ça vaut le coup

1. **Optimisation**: garder l'API Python, remplacer les modules à optimiser par du Rust. Par exemple Polars, pydantic-core, ...
2. **API polyglotte**: un noyau Rust, exposé partout Par exemple OpenDAL, xberg, ...
3. **Prototyper en Python puis migrer en Rust**: itérer +vite, faire performant quand on est stable
4. **Sandboxing, plugins**: controller du code Python non-fiable.Mais est-ce que WASM, ou les microVM ne sont pas mieux pour ça ?
5. **Rust dans CPython**: Rust for CPython (Python Language Summit 2026) | Python blog, "Bringing Rust into Python" | RustConf 2026, Rust for CPython | Github

Le menu parfait ?

1. **Profiler** — choisir un coût mesuré, comparer en `--release`.
2. **Batcher** — une petite API, du travail utile à chaque appel.
3. **Détacher** — laisser Python avancer pendant les longs traitements Rust.

→ **Python garde l'ergonomie ; Rust prend le travail lourd.**

Références

Merci !

Des questions ?

- **Outils** pyo3.rs, maturin.rs, écosystème PyO3, pyo3-async-runtimes, rust-numpy, py-spy, uv cache-keys
- **Les talks de David Hewitt**, mainteneur PyO3 PyO3 in depth, Free-threaded Python, 5 years of Rust in Python, ITW - From Python to Rust and Back Again
- **Le code de cette conf** github.com/ilaborie/rust-python-talk slides